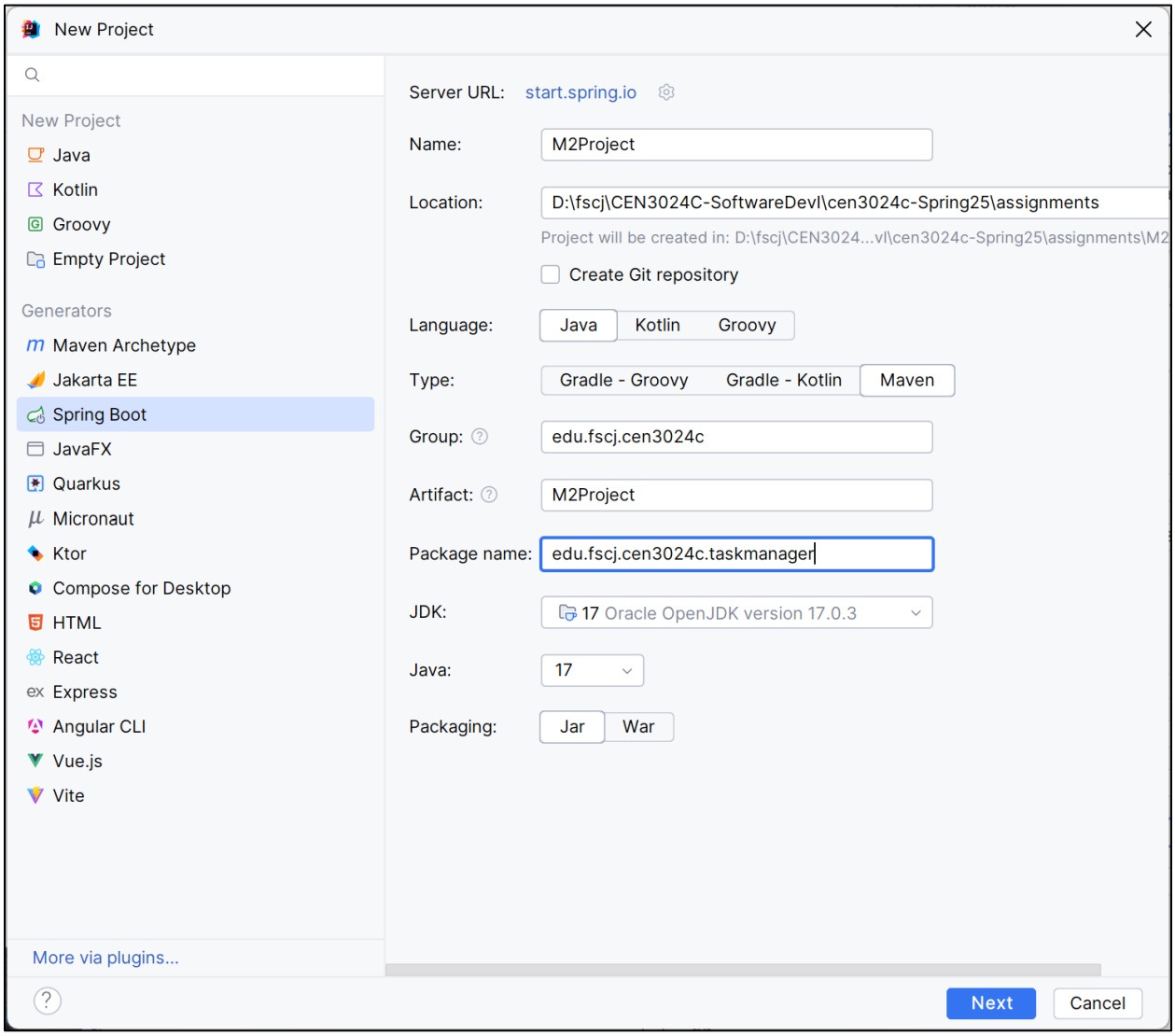
CEN3024C Module 2 Project

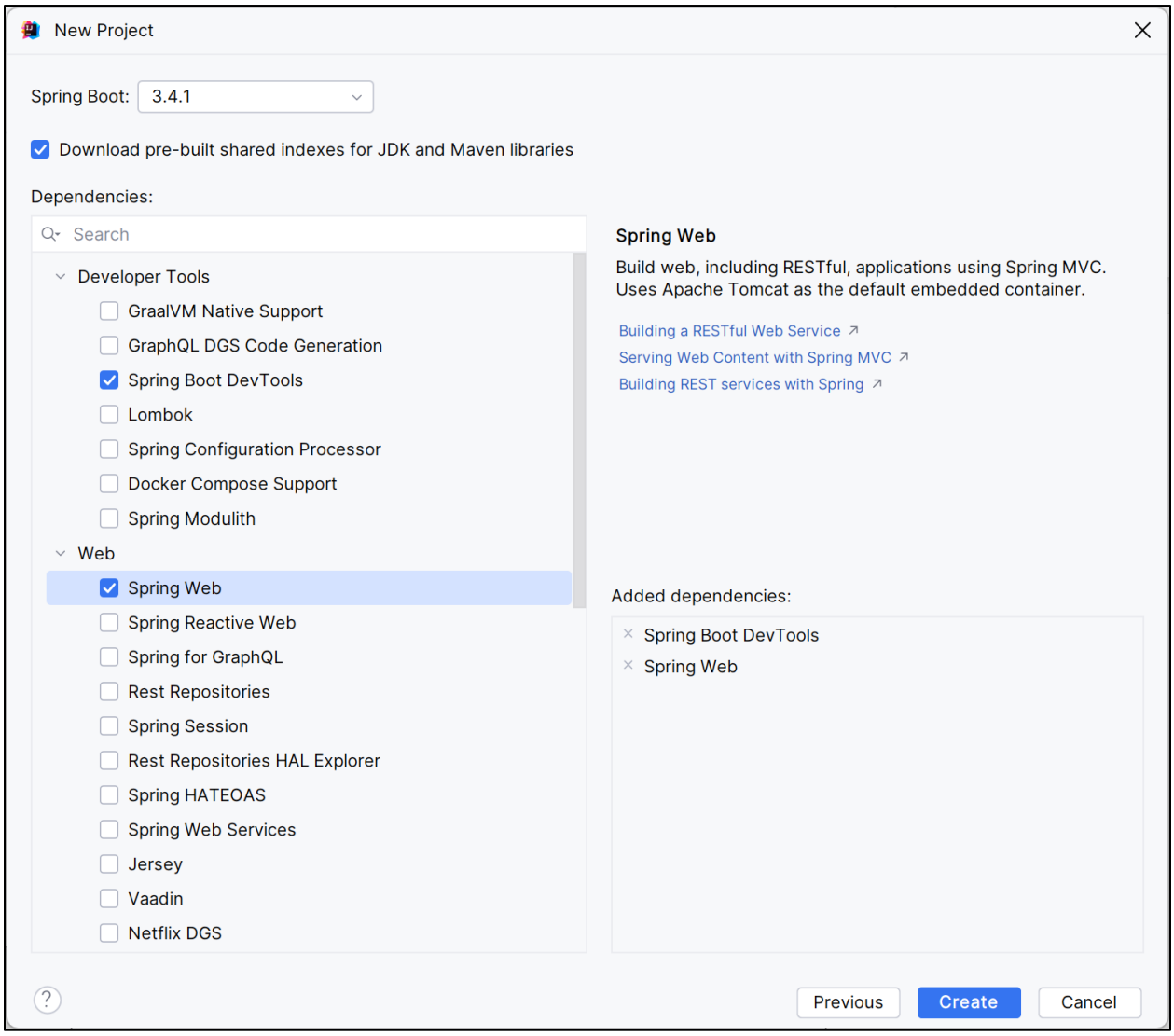
For this project, we will use the Spring Boot Initializr project generator to set up a simple task management application. The application will store tasks in an in-memory data structure. This project will give you hands-on practices in managing application data, implementing a service layer, and creating REST API endpoints.

- Start by creating a new IntelliJ project and select the Spring Boot generator. Select Java and Maven. Your group should be the "upper level" of your package (e.g., edu.fscj.cen3024c) and the package should extend that to indicate the project (taskmanager).
- Select Java 17 and Jar packaging.

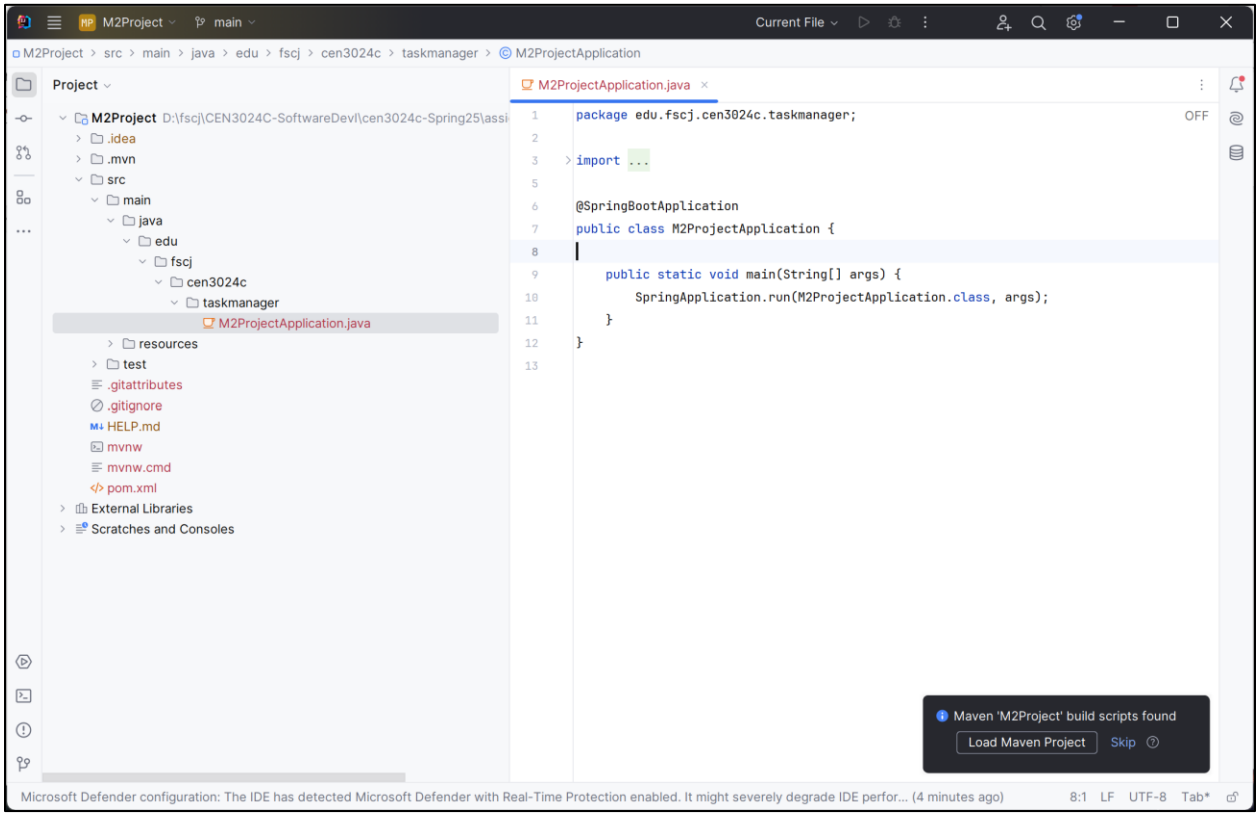


After clicking "Next" you will see the Spring Boot Dependencies page; select the following dependencies:

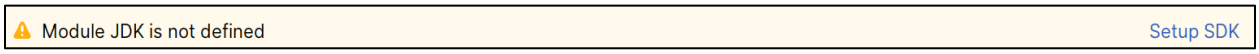
Developer Tools/Spring Boot DevTools
Web/Spring Web

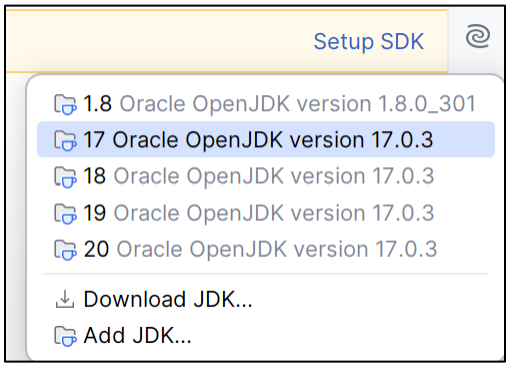


Click "Create" and IntelliJ creates the application class for you:



Note: if you see the "Module JDK is not defined", click "Setup SDK" and select the Java 17 JDK (do not select a different version, we are using 17 in this course).





Implementation:

1. Task Class (Model)

Create a Task class to represent the tasks. This will not be linked to a database but will act as the data model in the application.

```
package edu.fscj.cen3024c.taskmanager;

public class Task {
  private Integer id;
  private String title;
  private String description;
  private String status; // PENDING, IN_PROGRESS, COMPLETED
  private String dueDate; // YYYY-MM-DD

  // Constructors, getters, and setters
}
```

2. TaskService Class

Implement a TaskService class to handle business logic. Use an in-memory HashMap to store tasks.

```
package edu.fscj.cen3024c.taskmanager;

import org.springframework.stereotype.Service;

import java.util.*;

@Service
public class TaskService {
  private final Map<Integer, Task> taskMap = new HashMap<>();
  private int currentId = 1;

  public List<Task> findAll() {
    return new ArrayList<>(taskMap.values());
  }

  public Task findById(Integer id) {
    if (!taskMap.containsKey(id)) {
```

```
            throw new TaskNotFoundException(id);
        }
        return taskMap.get(id);
    }

    public Task save(Task task) {
        if (task.getId() == null) {
            task.setId(currentId++);
        }
        taskMap.put(task.getId(), task);
        return task;
    }

    public void deleteById(Integer id) {
        if (!taskMap.containsKey(id)) {
            throw new TaskNotFoundException(id);
        }
        taskMap.remove(id);
    }
}
```

3. TaskNotFoundException Class

Add a custom exception for when a task is not found.

```
package edu.fscj.cen3024c.taskmanager;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND)
public class TaskNotFoundException extends RuntimeException {
    public TaskNotFoundException(Integer id) {
        super("Task not found with id " + id);
    }
}
```

4. TaskController Class

Create a controller to expose REST endpoints for task management.

```
package edu.fscj.cen3024c.taskmanager;


import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/tasks")
public class TaskController {

  @Autowired
  private TaskService taskService;

  @GetMapping
  public List<Task> getAllTasks() {
    return taskService.findAll();
  }

  @GetMapping("/{id}")
  public Task getTaskById(@PathVariable Integer id) {
    return taskService.findById(id);
  }

  @PostMapping
  public Task createTask(@RequestBody Task task) {
    return taskService.save(task);
  }

  @PutMapping("/{id}")
  public Task updateTask(@PathVariable Integer id, @RequestBody Task task) {
    Task existingTask = taskService.findById(id);
    existingTask.setTitle(task.getTitle());
    existingTask.setDescription(task.getDescription());
    existingTask.setStatus(task.getStatus());
    existingTask.setDueDate(task.getDueDate());
```

```
        return taskService.save(existingTask);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteTask(@PathVariable Integer id) {
        taskService.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```

Testing the API:

Use cURL to test the REST API endpoints. Provide screenshots of each command's results as part of the project submission.

1. **Create a Task**:

(Windows)

```
curl -X POST http://localhost:8080/tasks ^
-H "Content-Type: application/json" ^
-d "{\"title\": \"New Task\", \"description\": \"Task description\", \"status\": \"PENDING\", \"dueDate\": \"2024-09-15\"}"
```

(Linux/Max)

```
curl -X POST http://localhost:8080/tasks \
-H "Content-Type: application/json" \
-d "{\"title\": \"New Task\", \"description\": \"Task description\", \"status\": \"PENDING\", \"dueDate\": \"2024-09-15\"}"
```

2. **Get All Tasks**:

```
curl -X GET http://localhost:8080/tasks
```

3. **Get One Task** (replace "id" with a specific id, usually 1)

```
curl -X GET http://localhost:8080/tasks/{id}
```

4. **Update a Task** (replace "id" with a specific id, usually 1)

(Windows)

```
curl -X PUT http://localhost:8080/tasks/1 ^
-H "Content-Type: application/json" ^
-d "{\"title\": \"Updated Task\", \"description\": \"Updated description\", \"status\": \"COMPLETED\", \"dueDate\": \"2024-10-01\"}"
```

(Linux/Mac)

```
curl -X PUT http://localhost:8080/tasks/{id} \
-H "Content-Type: application/json" \
-d "{\"title\": \"Updated Task\", \"description\": \"Updated description\", \"status\": \"COMPLETED\", \"dueDate\": \"2024-10-01\"}"
```

5. **Delete a Task** (replace "id" with a specific id, usually 1):

```
curl -X DELETE http://localhost:8080/tasks/{id}
```

Submission:

- Upload the completed project, including the Task model, service, controller, and any other files created.
- Include screenshots of the cURL results in your assignment repository.